Assignment 2: Unsupervised Learning
Name: Gulzhas Mailybayeva
Student ID: EC25184
Module: ECS708U/ECS708P-Machine Learning

1. Introduction

The goal of this assignment was to gain an understanding of the Mixture of Gaussians (MoG) model for density estimation. We used the Peterson and Barney's dataset of vowels formant frequencies and Expectation-Maximization (EM) algorithm to fit MoG models to a dataset of phoneme features. The investigation covered initial data exploration, model stability, using MoG as a generative classifier, and diagnosing and solving the "singularity problem" that can arise during training.

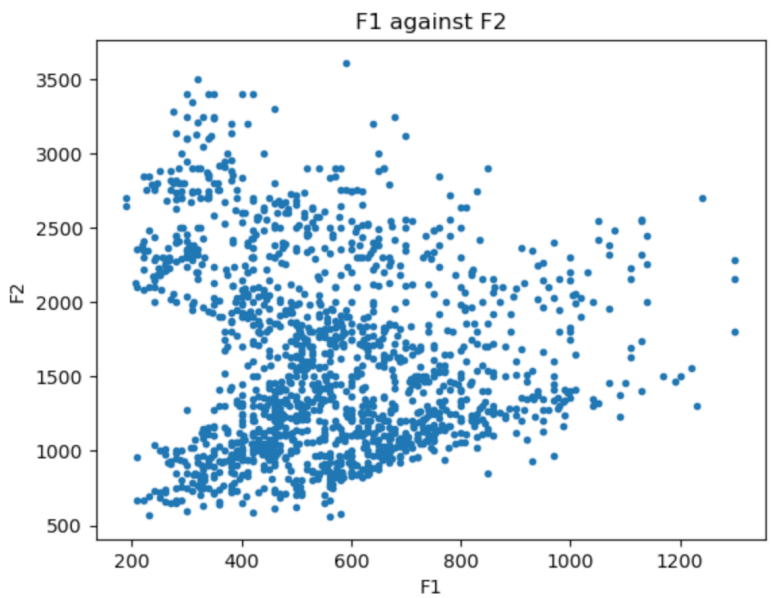
2. Data Exploration and Model Stability

Q1: Initial Data Visualization

The Peterson & Barney vowel dataset, consisting of F1 and F2 phoneme features, was first plotted on a 2D scatter plot to observe any initial structure. Each point corresponds to one recorded vowel token.

Figure 1: Scatter plot of F1 vs F2 features for the entire dataset.

```
### your code here
plt.figure()
plt.scatter(f1, f2, s=9)
plt.xlabel('F1'); plt.ylabel('F2'); plt.title('F1 against F2')
plt.show()
```



As seen in Figure 1, the data is not uniformly distributed. Some clusters (e.g. high front vowels with low F1 and high F2) are well separated, while others (mid / low vowels) overlap more. There are several clear, dense groupings of points, suggesting that the data is a "mixture" of different underlying distributions.

Q2: Training a MoG with K = 3 on phoneme_id = 1

A MoG model with ***K=3*** components was trained on the data for ***phoneme_id=1***. To test the stability of the EM algorithm. EM was initialized with random component means (chosen from data points), diagonal covariance matrices, and uniform mixture weights.

Q3: EM stability and label switching

To investigate the stability of EM, I run K=3 training on *phoneme_id=1* **four times** with different random initializations.

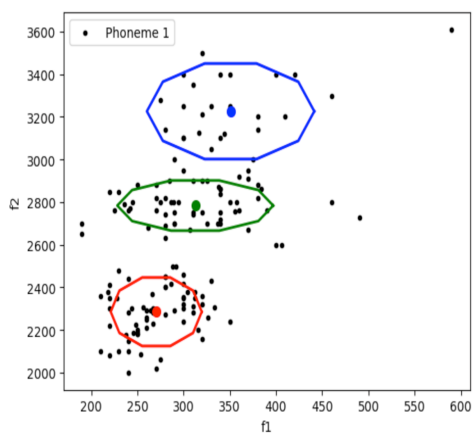
Observation:

- In every run, the log-likelihood increased monotonically and converged to the same final value, indicating that EM consistently found the same mode of the likelihood for this dataset.
- The cluster shapes and locations in the F1-F2 space were also the same runs: one component always covered the low F1/high F2 region, another the intermediate region etc.

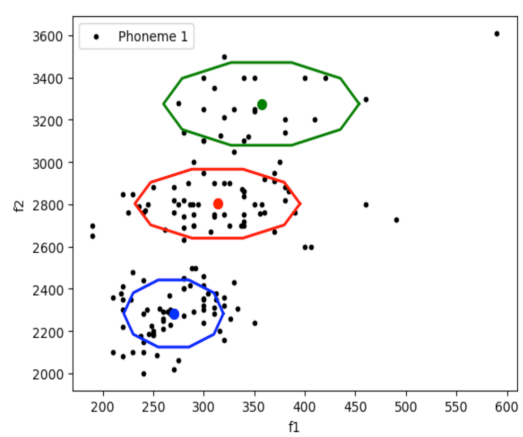
However, the assignment of these parameters to a specific index (and thus, a specific color in the plot) was arbitrary and changed between runs. For example, in three runs, the cluster at [270, 2285] was assigned as the "green" circle, while in the fourth run, it was assigned as the "red" circle.

This "label switching" is a key observation. It demonstrates that while the random initialization affects the label of a cluster, for this dataset, the EM algorithm is stable and robust, consistently finding the same global optimum (the three distinct clusters) regardless of the starting point.

Figure 2: MoG Train Results with K=3 and p_id=1




```
Finished.

[[ 270.3952  2285.4653 ]
 [ 312.59125 2783.898  ]
 [ 350.8446  3226.3394 ]] [[[ 1213.73843494     0.        ]
  [    0.         14278.42029959]]

 [[ 3562.59743766     0.        ]
  [    0.          7657.84897186]]

 [[ 4102.87537489     0.        ]
  [    0.         27829.54222036]]]
```

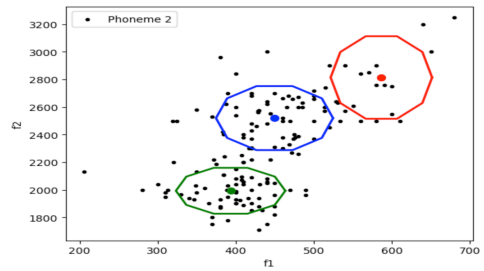



```
Finished.

[[ 313.61365 2803.1616 ]
 [ 356.99387 3275.2922 ]
 [ 270.17682 2282.9385 ]] [[[ 3339.96470564     0.        ]
  [    0.         14713.32772371]]

 [[ 4689.90493449     0.        ]
  [    0.         21213.24721313]]

 [[ 1216.32884772     0.        ]
  [    0.         13923.59716192]]]
```

Q4: Training MoG models for phoneme_id = 2 and for both phonemes

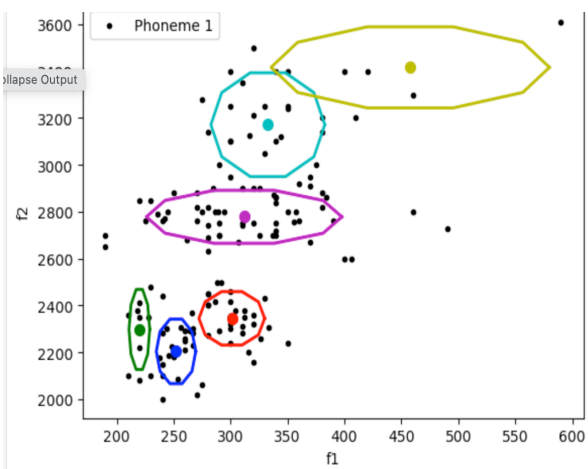
Following this, models were successfully trained for ***phoneme_id=2 (K=3***) and for both phonemes (***phoneme_id=21, phoneme_id=2)*** with ***K=6*** components, saving *(save_model(mu, s, p))* the parameters for later use and report submission in .zip file.

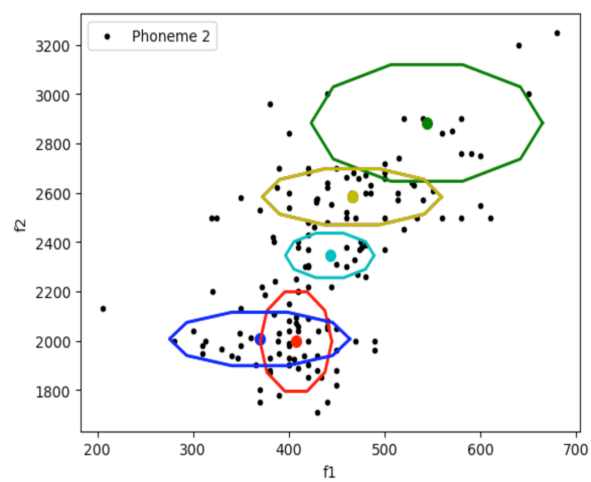
Figure 3: MoG Train Results with K=3 and p_id=2




```
Finished.

[[ 586.56946 2814.204  ]
 [ 393.4532  1993.0856 ]
 [ 449.67908 2519.6936 ]] [[[ 2111.34948331     0.        ]
  [    0.         49424.394191851]

 [[ 2454.89378393     0.        ]
  [    0.         15264.2754882 ]]

 [[ 2809.54480457     0.        ]
  [    0.         29719.62014968]]]
```

Figure 4: MoG Train Results with K=6 and p_id=1 and p_id=2






```
Finished.

[[ 301.07275 2345.0056 ]
 [ 219.87886 2297.635  ]
 [ 252.07738 2203.9734 ]
 [ 332.71353 3171.6624 ]
 [ 311.8664  2778.346  ]
 [ 457.79926 3416.4011 ]] [[[  417.61068598     0.        ]
  [    0.          7209.52881181]]

 [[   41.36495107     0.        ]
  [    0.         16137.45609384]]

 [[  150.10132022     0.        ]
  [    0.         10443.90028967]]

 [[ 1251.67390313     0.        ]
  [    0.         27163.85900134]]

 [[ 3684.04765009     0.        ]
  [    0.          7056.11999316]]
```

```
Finished.

[[ 407.43735 1996.615  ]
 [ 544.05237 2883.2632 ]
 [ 369.55658 2006.8536 ]
 [ 442.71964 2345.78   ]
 [ 465.91376 2583.5217 ]
 [ 465.91376 2583.5217 ]] [[[  702.79244282     0.        ]
  [    0.         22515.42606612]]

 [[ 7300.43101584     0.        ]
  [    0.         30936.15312187]]

 [[ 4435.10872786     0.        ]
  [    0.          6497.23826571]]

 [[ 1071.18804304     0.        ]
  [    0.          4518.2004669 ]]

 [[ 4372.94270331     0.        ]
```

3. MoG as a Generative Classifier

Q5-Q6: Classification and Accuracy

Two generative classifiers were built to discriminate between phoneme_id=1 and phoneme_id=2. The classification used the Maximum Likelihood (ML) criterion, comparing the likelihood of a data point **x** being generated by each model:

$$p(\mathbf{x} \mid c) = \sum_{k=1}^{K} \pi_k^{(c)} \, \mathcal{N}(\mathbf{x} \mid \mu_k^{(c)}, \Sigma_k^{(c)}),$$

where $\pi_k^{(c)}$ are the mixture weights, $\mu_k^{(c)}$ the component means, and $\Sigma_k^{(c)}$ the covariance matrices for that class.

The **classification rule** is Maximum Likelihood (ML):

$$\hat{c}(\mathbf{x}) = \arg\max_{c \in \{1,2\}} \; p(\mathbf{x} \mid c),$$

or equivalently (and more numerically stable):

$$\hat{c}(\mathbf{x}) = \arg\max_{c \in \{1,2\}} \; \log p(\mathbf{x} \mid c).$$

For each test point **x**, I compute $\log p(\mathbf{x} \mid c = 1)$ and $\log p(\mathbf{x} \mid c = 2)$ using the stored MoG parameters and assign **x** to the class with the higher log-likelihood.

The two classifiers used models with ***K=3*** and ***K=6*** components, respectively.

For each classifier, I computed the **misclassification error** and **accuracy** on the test data:

$$\text{error} = \frac{\#\text{ misclassified points}}{\#\text{ total points}}, \text{accuracy} = 1 - \text{error}.$$

Model Components	**Miss-classification Error**	**Accuracy**
K=6	0.0461	95.39%
K=3	0.0493	95.07%

The resulting accuracies are as follows, K =6 model shows slightly higher accuracy, but the difference is small. This suggests that adding extra components gives a bit more flexibility to model the class-conditional densities.

Figure 5: Q5 code snippet

```
# Initialize array Z_phoneme_1 that will get the predictions of each Gaussian on e
Z_phoneme_1 = np.zeros((N,k)) # shape Nxk

# get predictions
###your code here -- get_predictions(mu, s, p, X):
Z_phoneme_1 = get_predictions(mu, s, p, X)
L1 = np.sum(Z_phoneme_1, axis = 1)
print(f"Got likelihoods from Model 1 (p_id={p_id}).")


# Get predictions on both phonemes 1 and 2, from a GMM with k components, pretrain
p_id = 2
###your code here
npy_filename = 'data/GMM_params_phoneme_{:02}_k_{:02}.npy'.format(p_id, k)
if os.path.isfile(npy_filename):
    GMM_parameters_phoneme_2 = np.load(npy_filename, allow_pickle=True)
    GMM_parameters_phoneme_2 = np.ndarray.tolist(GMM_parameters_phoneme_2)
else:
    raise Exception('File {} does not exist.'.format(npy_filename))
mu = GMM_parameters_phoneme_2['mu']
s = GMM_parameters_phoneme_2['s']
p = GMM_parameters_phoneme_2['p']

Z_phoneme_2 = get_predictions(mu, s, p, X)
L2 = np.sum(Z_phoneme_2, axis = 1)
print(f"Got likelihoods from Model 2 (p_id={p_id}).")

# classify each point using Maximum Likelihood
### your code here
y_pred = np.zeros(N)
y_pred[L1 > L2] = 1
y_pred[L2 > L1] = 2
y_pred[L1 == L2] = 1
print("Classification complete.")

# calculate accuracy
### your code here
# Create the ground truth labels based on the concatenation order
y_true = np.concatenate((np.ones(len(inds_1)), np.ones(len(inds_2)) * 2))
errors = np.sum(y_pred != y_true)
miss_classification_error = errors / N
```

Q7: Decision Boundary Analysis

To understand models' classification performance, I generated decision boundary plots for both classifiers.

Figure 6: *K=3* Classifier Decision Boundary

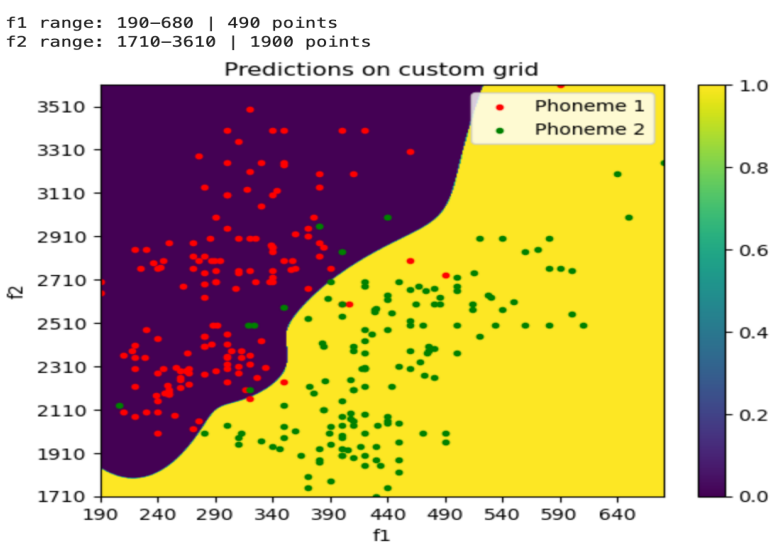
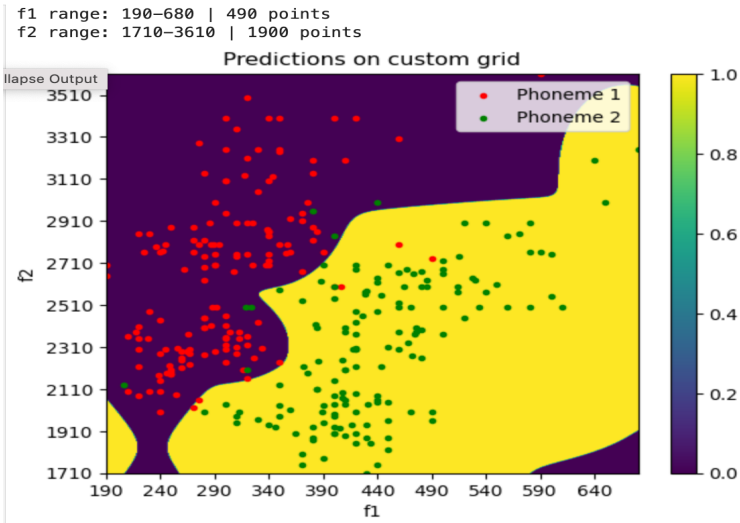


Figure 3: K=6 Classifier Decision Boundary



The reason for the accuracy difference is immediately clear from these plots:

- The K = 3 decision boundary is relatively **smooth** and simple. There is a single main curve separating the two classes, roughly aligned with the area where the two phoneme clouds overlap in the scatter plot. This boundary captures the broad structure of the data, but it cannot model finer details: some points near the overlap region are on the "wrong" side of the boundary and end up misclassified.
- The K = 6 classifier produces a **more flexible, curved boundary**. Because the mixture has more components, it can better follow the shapes of the data clouds for each phoneme. In the overlapping region, the boundary bends around the clusters more tightly, carving out smaller regions for each class and reducing the number of ambiguous points that fall on the wrong side.

Q8: The Problem of Collinear Features

A new 3D dataset was created where the features were collinear:

$$\mathbf{X} = [F1,\ F2,\ F1 + F2].$$

The MoG training algorithm was then run on this 3D data.

Observation: The training failed immediately, producing a RuntimeWarning: divide by zero followed by a ValueError: Input contains infinity.

This is the singularity problem. Because the third feature is a perfect linear combination of the first two, all the data points lie on a flat 2D plane within the 3D space. The EM algorithm attempts to fit a 3D Gaussian ellipsoid to this flat data, causing the covariance matrix s to collapse and have no variance in one dimension. A matrix with no variance is "singular," meaning its determinant is 0. The get_predictions() function must calculate 1 / det(s), which results in a division by zero and crashes the algorithm.

Q9: Suggesting and Implementing a Solution

The most common way to overcome Q8 problem is Covariance Regularization (or "jitter"). This was the method I chose to implement.

During the M-step of EM, each Gaussian component's covariance matrix is updated based on the weighted sample covariance of the data. When the data are collinear or nearly low rank, the raw covariance estimate Σ_k^{raw}becomes singular, meaning:

- $\det(\Sigma_k^{\text{raw}}) = 0$,
- Σ_k^{raw}cannot be inverted,
- and the Gaussian log-likelihood becomes undefined.

To prevent this, I modified the covariance update to:

$$\Sigma_k \leftarrow \Sigma_k^{\text{raw}} + \epsilon I,$$

where $I$is the identity matrix and $\epsilon$is a small positive constant. In my implementation I used:

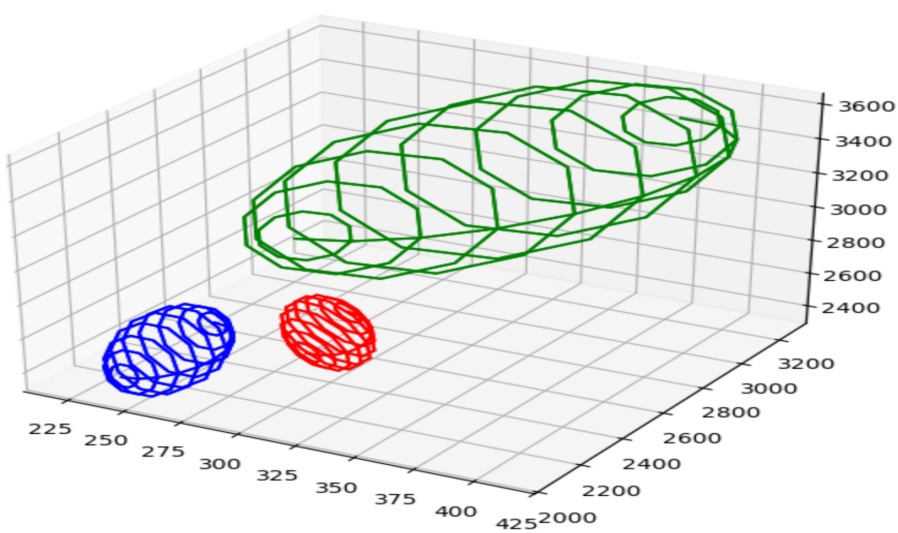
$$\epsilon = 10^{-6},$$

which is large enough to stabilize the matrix but small enough not to distort the learned covariance structure.I added this regularization in two places:

1. **Inside the EM M-step**, immediately after computing the covariance.
2. **Inside the `get_predictions` function**, ensuring that likelihood evaluation never fails due to numerical issues.

With these changes: EM no longer crashed on the 3D collinear dataset; The final Gaussian components formed very "flat" ellipsoids, correctly reflecting that the data lie on a lower-dimensional plane; All values in the outputs were finite; no `inf` or `NaN` appeared;

Figure 4: The MoG model successfully fitting to the collinear 3D data after applying covariance regularization.



4. Conclusion

This lab was a practical exploration of the MoG model. The key takeaways were:

1. **MoG is effective:** The model was highly effective at identifying the underlying cluster structure in the 2D phoneme data.
2. **EM was stable for random initializations:** For both phoneme classes, EM consistently converged to the same likelihood value and the same cluster geometry, with component labels permuted only due to the inherent label-switching symmetry in mixture models.
3. **Decision boundaries show why performance differ:** The K=3 produced smoother and simpler boundaries, while K=6 model produced complex cluster shapes. This explains the modest accuracy improvement.
4. **The Singularity problem is real and important:** I was able to replicate the singularity crash by using collinear data.
5. **Regularization is the Solution:** I successfully solved the singularity problem by implementing covariance regularization, which made the algorithm robust and allowed it to converge.

5. Note on use of AI tools

I have used AI tools for specific tasks to enhance my understanding of the matter and refine my work. These tools primarily assisted in:

- **Deepening Understanding:** Utilizing AI to learn about and understand the **MoG** (Matter of concern/guidance) in depth.
- **Debugging Code:** Identifying and resolving issues in my code.
- **Proofreading and Editing:** Ensuring the clarity, coherence, and grammatical accuracy of the report.

The final outputs reflect my own work and critical judgment.

Q9 Code implementation

```
##########################################
# Write your code here
import numpy as np
import scipy.linalg
from sklearn.preprocessing import normalize
import matplotlib.pyplot as plt
import os
from IPython import display
from mpl_toolkits.mplot3d import Axes3D # Need this for 3D plot

# --- 1. Load Data ---
# Note: This assumes 'PB_data.npy' is in the same directory
```

```
try:
    data = np.load('PB_data.npy', allow_pickle=True).item()
    f1 = data['f1']
    f2 = data['f2']
    phoneme_id = data['phoneme_id']
except FileNotFoundError:
    print("Error: PB_data.npy not found.")
    # Create dummy data as a fallback to prevent crashing
    f1 = np.random.rand(100) * 100
    f2 = np.random.rand(100) * 100
    phoneme_id = np.concatenate((np.ones(50), np.ones(50)*2)).astype(int)

# --- 2. Q9 Implementation (Q8 code with the fix) ---
# This code assumes the helper functions (get_predictions, plot_data_3D, plot_gaussians)
# have already been defined in your notebook's global scope
# and that get_predictions has the Q9 (jitter) fix.

print("--- Starting Q9 (3D GMM with Regularization) ---")

p_id = 1 # id of the phoneme that will be used (e.g. 1, or 2)
k = 3 # number of GMM components

# Create the 3D data
X_full_3D = np.zeros((len(f1), 3))
X_full_3D[:, 0] = f1
X_full_3D[:, 1] = f2
X_full_3D[:, 2] = f1 + f2

# Filter for the chosen phoneme
X = X_full_3D[phoneme_id == p_id, :]


# --- Plot initial 3D data ---
fig = plt.figure(figsize=(8, 7)) # Make figure a bit bigger
ax1 = plt.axes(projection='3d')
title_string = 'Phoneme {}'.format(p_id)
plot_data_3D(X=X, title_string=title_string, ax=ax1)
```

```
plt.show()

# --- Initialization ---
N, D = X.shape
p = np.ones((k))/k
random_indices = np.floor(N*np.random.rand((k)))
random_indices = random_indices.astype(int)
mu = X[random_indices,:] # shape kxD
s = np.zeros((k,D,D)) # shape kxDxD
n_iter = 150

# initialize covariances
for i in range(k):
    cov_matrix = np.cov(X.transpose())
    s[i,:,:] = cov_matrix/k

Z = np.zeros((N,k)) # shape Nxk

# Define the fix parameters
identity_matrix = np.eye(D)
regularization_value = 1e-6

print("Starting EM algorithm (with Q9 singularity fix)...")

##################################
# run Expectation Maximization algorithm for n_iter iterations
for t in range(n_iter):
    if (t+1) % 10 == 0 or t == n_iter - 1:
        print('Iteration {:03}/{:03}'.format(t+1, n_iter))

    # Do the E-step (using the fixed get_predictions)
    Z = get_predictions(mu, s, p, X)
    Z = normalize(Z, axis=1, norm='l1')

    # Do the M-step:
    for i in range(k):
        mu[i,:] = np.matmul(X.transpose(), Z[:,i])/np.sum(Z[:,i])
```

```
        ##########################################################
        # We will fit Gaussians with full covariance matrices:
        mu_i = mu[i,:]
        mu_i = np.expand_dims(mu_i, axis=1)
        mu_i_repeated = np.repeat(mu_i, N, axis=1)

        term_1 = X.transpose() - mu_i_repeated
        term_2 = np.repeat(np.expand_dims(Z[:,i], axis=1), D, axis=1) * term_1.transpose()

        # Calculate the original covariance
        s_i_raw = np.matmul(term_1, term_2)/np.sum(Z[:,i])

        ##########################################################
        # Q9 FIX (JITTER): Add the regularization value
        s[i,:,:] = s_i_raw + regularization_value * identity_matrix
        ##########################################################

        p[i] = np.mean(Z[:,i])

    # Plotting (show fewer plots to speed up)
    if (t+1) % 15 == 0 or t == n_iter - 1:
        # Re-create figure object for animation
        fig = plt.figure(figsize=(8, 7))
        ax1 = plt.axes(projection='3d')
        plot_gaussians(ax1, 2*s, mu, X, 'Phoneme {} (Iter {})'.format(p_id, t+1))
        plt.show()

print("EM Algorithm finished successfully.")
```